LOOKER LAB · CAPEX SYSTEMS ONBOARDING · DATA INTEGRITY PROGRAM MANAGER

Work one program from the tracker to the scorecard

Role. You make the inputs accurate when a program moves off Excel trackers onto the CapEx system of record, and you do the change with GSM, TPM, AME, the contract manufacturers, and Finance without escalating it.
Tool. Looker on Google Cloud, the product with LookML. Excel is the must-have. This lab is how you explain the scorecard, not a claim that you have shipped LookML.
Time. Path A is about 45 minutes on this machine. Path B is a real Looker or Looker Studio session and takes longer.

Illustrative, and labeled that way. Not Apple's system and not a program you ran. People, suppliers, and sites are lab labels. Every rate below is from `python3 run_local.py` against these files. The lived numbers you are allowed to put next to it are named where they appear: a counted-rows go/no-go and "Negative AFS clean at T+1"; 44 requirement rows across 23 systems; an issues log of 331 go-live, 77 fast-follow, 36 enhancement; Austria data quality about 55% to about 99% within a quarter; Shipment Visibility events-in versus events-out; a Zendesk-to-Snowflake requirements document where you wrote seven metric definitions and named Looker SQL Runner. You did not write the Looker layer on that pipeline.

Part 0 — What we are doing and why

A test-station line, `TS-LINE-RAMP`, is still living in two contract-manufacturer trackers. The team's job is to get that program onto the system of record with inputs someone can defend: the PO line matches an asset, the placed-in-service date is the day depreciation starts, the site code is the site the tool is actually at, a reused fixture points at the asset it came from, and each of those fields has a named owner. The scorecard is how you see that without opening the tracker. The queue is how you close it without a side file.

Two other programs are already loaded and clean, on purpose. `PACK-CELL-EXP` has 32 lines. `FIXTURE-REUSE-24` has 28, including the twelve source assets the ramp reuses. If you forget the program filter, reconciliation is 98 of 100, which is 98.0%. The ramp alone is 38 of 40, which is 95.0%. The blend looks finished. The program you are onboarding is not.

What the lab proves. You can read a messy tracker, publish a tolerance before you load, quarantine what fails a structural rule, and still find the business defects the structural rules did not cover. You can define "reconciled" once, show three other formulas that print a healthier number, and refuse them. You can put an owner on each field, age the open defects, and show that fixing one record closes the exception and does not repair first-time-right.

The final output is the data-integrity scorecard for `TS-LINE-RAMP`, as of 2026-03-16, plus the exception list behind it.

TILE	RESULT FROM THE SQL RUN
Reconciliation	38 / 40 = 95.0%. Alert fires. The proposed line is under 98%.
Exception aging, open	7 in 0–7 days, 7 in 8–30, 7 in 31+. Twenty-one open, all go-live.
Field completeness by owner	Placed-in-service 28 / 40 = 70%, owner Finance. Reuse 12 / 12. Site 100% complete and not 100% valid.
First-time-right	21 / 40 = 52.5%.
Person-dependent fields	3. Placed-in-service (Morgan Ellis), program allocation (David Cho), reuse source (Elena Vasquez).

That output is this role. The accuracy of the inputs is the job. The owner on the field is how you ask for a correction without escalating. The playbook rule at the end is how the next tracker does not repeat the miss. A chart with no owner is a status meeting. This scorecard is a close-out list.

WHAT YOU SAY, IF THEY ASK WHAT YOU ARE ABOUT TO SHOW

I would walk one program off the tracker and onto a scorecard where reconciled means one thing, every blank has an owner, and a late fix does not pretend the first entry was right.

Part 1 — Two paths, honestly

Path A — this machine, about 45 minutes

You do not need Looker, a cloud account, or a card. DuckDB runs the same checks the answer key was built from. The dashboard you can hold up is `dashboard_mock.html`, drawn from those query results. It is a static page, not a Looker session. Say that in the first sentence if you show it.

```
cd /workspace/apple/looker-lab
python3 generate_data.py
python3 run_local.py
```

`generate_data.py` rewrites the CSVs from a fixed seed. You can skip it; the files are already there. `run_local.py` loads them, runs `sql/02` and `sql/03`, writes `data/expected_outputs/`, applies the step-9 fix into `data/expected_outputs/after_fix/`, and leaves `/tmp/lookerlab-scratch/lab.duckdb` on the open queue so a query matches the answer key. Open `dashboard_mock.html` in a browser. Diff any CSV you compute against `expected_outputs/`.

If DuckDB is missing: `pip install duckdb`. The SQL that creates the tables also exists as BigQuery in `sql/00_create_tables.sql`. The file you execute locally is `sql/00_create_tables_duckdb.sql`.

Path B — a real Looker, or Looker Studio if that is what you have

1. **BigQuery sandbox.** Google's sandbox is free and does not ask for a card. It has storage and query caps this lab will not touch. Confirm the current cap on Google's page before you depend on it. Create a dataset named `capex`.
2. **Load the CSVs** with the column types in `sql/00_create_tables.sql`. `po_line_id` is STRING. If the loader guesses a number, `00017` becomes `17` and every join fails. Do not load the raw trackers into the fact tables. They are the file the gate reads. `sql/01_load.sql` has the `bq load` shape.
3. **Connect Looker.** Looker, the product with LookML, needs a Google Cloud Looker instance or a trial. There is no free Looker core that simply attaches to the sandbox. **Looker Studio is a different product.** It is free, it can sit on these BigQuery tables, and it can draw the same five tiles. It does not run the LookML in `lookml/`. If they say "Looker," ask which one, then stay on the one they named.
4. **If you have Looker:** new project, connection name whatever your instance uses (the model file says `capex_bigquery_placeholder`), paste the LookML, validate, and explore. Set user attribute `program_ids` from `user_program_map.csv`. Jordan's value is `TS-LINE-RAMP` only.
5. **If you have Looker Studio:** build the five tiles as five charts on the BigQuery tables, with the SQL from `sql/03_metrics.sql` as the definition you refuse to fork. You will not have an access filter or a Git-backed measure. Say that. The grain and the tolerance are still yours.
6. The checks in `sql/02` and `sql/03` are DuckDB, because that is what was executed. On BigQuery, `date_diff('day', opened, as_of)` becomes `DATE_DIFF(as_of, opened, DAY)`, and the CSV reader becomes an external table. The counts do not change.

> **WHAT YOU SAY ABOUT THE TOOL**
>
> I have not shipped LookML. On the Snowflake pipeline I wrote seven metric definitions and named Looker SQL Runner as the surface. The infrastructure team built the pipeline. I can specify the grain, the owner, and the tolerance, and I would want your Looker developer on the model until I have one in production.

Part 2 — Step by step

1. Inspect the raw trackers and list the defects by eye

WHAT. Open `data/raw_cm_tracker_siteA.csv` and `data/raw_cm_tracker_siteB.csv` in a text editor, not by saving them out of Excel. Excel will turn `00017` into `17`.

WHY IT MATTERS. The must-have skill is Excel, and the failure mode of Excel is a silent type change. You list what a person can see before you trust a loader. You will not see all of it. That is why the checks exist.

What is visible without SQL:

- Two header languages. Site A's header has trailing spaces: `PO Line ` , `Site ` , `Qty ` , `Unit Cost ` , `Notes ` . Site B uses `po_line`, `PO#`, `Location`, `date received`.
- Three date shapes: `2026-01-21`, `01/08/2026`, `12-Feb-2026`. One impossible date: `13/32/2026`.
- Site text: `A`, `Site A`, `SITE-A`, `site a`, `Site B`, `SITE-B`, `B`. Line `00024` sits in the site A file and says `Site B`. Line `00026` says `PLANT-01`. Line `00025` says `A` in the site B file.
- Line `00018` appears twice, quantity 2, two serials.
- `SN-DUP-4421` on lines `00020` and `00021`. `SN-DUP-4422` on `00022` and `00023`.
- Blank in-service cells. You may not count all twelve by eye. The SQL says twelve.
- Line `00019`: in-service `11-Feb-2026`, received `2026-02-18`.
- Lines `00039` and `00040`: no asset tag, no received date.
- Row `00901` with site `SITE-Z`. A row with a blank PO line. A fully blank row at the bottom of the site B file.
- Trailing space on serial `SN-TS-0005 ` and asset tag `AST-TS-0005 ` .

What you will not see: asset `AST-TS-0040`. It was keyed into the system of record and is on neither tracker.

EXPECTED. A written list in those words. The blank Excel row is not a defect and not a submission.

> **WHAT YOU SAY**
> I read the file before I load it, and I write down what I can see, because the loader will hide the rest inside a green row count.

2. Load

WHAT. Run `sql/00_create_tables_duckdb.sql` and `sql/01_load_duckdb.sql`, or just `python3 run_local.py`. The fact tables are the system of record. The equipment list is the authority for which PO lines exist. The trackers are not copied in as facts. Values are trimmed. Empty strings become null. Ids stay strings.

WHY IT MATTERS. A load that "succeeds" with the wrong type is a bad load. The counted-rows read after load is 100 PO lines, 100 assets, 21 exceptions, and 40 lines on `TS-LINE-RAMP`.

EXPECTED. `SELECT COUNT(*) FROM fact_po_line` → 100. `WHERE program_id = 'TS-LINE-RAMP'` → 40. `fact_asset` → 100. `fact_exception` → 21. `SELECT po_line_id FROM fact_po_line WHERE po_line_id = '00017'` returns the five-character id, not 17.

> **WHAT YOU SAY**
> I count the rows after the load against the count I published before it, and I check that the line id is still text.

3. Quality checks, and the counted-rows go / no-go

WHAT. Read `data/published_tolerance.csv` first. The rules are dated 2026-03-01. Then run `sql/02_quality_checks.sql`. The tolerance is not edited after you see the file.

WHY IT MATTERS. On the Sephora S/4 HANA work, the go / no-go was a counted-rows read, and the bar I still use the morning after a load is the one we called "Negative AFS clean at T+1": the business rules, not the file shape, have to be clean the next day. Here the structural gate can pass and the T+1 rules can still be red. Both results belong in the room.

CHECK	VALUE	BAR, PUBLISHED 2026-03-01	RESULT
Submitted tracker rows	43		
Quarantined	2		
Quarantine rate	2 / 43 = 4.65%	R3 ≤ 5%	Pass
Accepted rows	41		40 lines plus the second asset row on 00018
Distinct PO lines loaded	40	Equipment list 40	Pass
Missing list lines	0	R1 = 0	Pass
Duplicate PO line ids in the load	0	R2 = 0	Pass. The fan-out is not a duplicate id.
Decision	GO		The file may load. The book is not clean.

Quarantine detail, from `quarantine_rows.csv`: site A row `00901`, reasons `unknown_site` and `not_on_equipment_list` (the raw site is `SITE-Z`). Site B row with a blank PO line, reasons `blank_po_line` and `bad_po_date` (the raw date is `13/32/2026`). The blank artifact row is ignored. `tracker_vs_fact.csv` is 0 received-date mismatches and 0 linked-asset mismatches. The orphan is the one asset the tracker does not have.

What the structural gate did not stop, and the T+1 checks did:

CHECK	ROWS
Duplicate serials	2 serials, each on 2 assets. SN-DUP-4421 on AST-TS-0021 and AST-TS-0022. SN-DUP-4422 on AST-TS-0023 and AST-TS-0024.
Orphan	1. AST-TS-0040, no PO line, created by Jordan, site CM-B.
Site mismatch	3. 00024 PO CM-A / asset CM-B. 00025 PO CM-B / asset CM-A. 00026 PO CM-A / asset PLANT-01.
Placed-in-service before receipt	1. Line 00019, received 2026-02-18, date 2026-02-11, seven days early.
Referential, other	0 bad programs, sites, suppliers, owners, or reuse sources. The only referential failure is the orphan.

EXPECTED. Decision **GO** on the file, and a queue that is not empty. A gate you tighten after you see the quarantine rate is not a gate.

> **WHAT YOU SAY**
>
> I publish the tolerance before the file arrives. This file is a go on counted rows, and it is not clean: the rules I had not written yet are the queue.

4. Define the metrics once

WHAT. Read the view files in `lookml/` next to `sql/03_metrics.sql`. A **dimension** is something you group or filter by: program, site, owner, a date. You do not sum it. A **measure** is an aggregate: a count distinct, or a ratio of two of those. The **primary key** on `fact_po_line` is `po_line_id`. `count_distinct` of that key is a symmetric aggregate: Looker will not let the one-to-many asset join multiply it. `type: count` counts joined rows. A ratio you type by hand is not protected.

WHY IT MATTERS. Two spreadsheets with two formulas is how “reconciled” forks. On Shipment Visibility, the useful watch was events-in versus events-out, both sides, not a blended status. Same shape here: lines in, assets out. The requirements I wrote for Zendesk data in Snowflake froze seven metric definitions and named Looker SQL Runner. The acceptance gate on that pipeline was a parallel run within 1% for consecutive weeks. I did not write the LookML. This lab’s bar is the counted-rows tolerance, a different file. The habit is the same: one definition, written down before the tile.

Line `00018` is quantity 2, assets `AST-TS-0018` and `AST-TS-0019`. It is not a defect. It is the fan-out.

ON TS-LINE-RAMP	RESULT
Joined rows after the asset join	41

ON TS-LINE-RAMP	RESULT
Plain count of po_line_id	41. The line with two assets is counted twice.
Count distinct po_line_id	40
Count distinct asset_id on the join	39. The orphan is not on this join.
Line 00018 alone	plain count 2, distinct lines 1, assets 2

DEFINITION	RATE	CERTIFY?
Lines with at least one asset / lines	38 / 40 = 95.0%	Yes. The two in-transit lines stay in the denominator.
Assets on a line / lines	39 / 40 = 97.5%	No. The extra asset on 00018 hides one missing line.
Lines with an asset / lines that have a received date	38 / 38 = 100%	No. A default filter dropped 00039 and 00040.
Assets allocated to the program / lines	40 / 40 = 100%	No. The orphan fills the hole and is not a reconciliation.
First-time-right, all assets on the program	21 / 40 = 52.5%	Yes.
First-time-right, dropping the orphan	21 / 39 = 53.85%	No. The PO explore cannot see an asset with no line.
Reuse source filled / every asset	12 / 40 = 30%	No. The field applies to 12 reuse lines, and 12 of 12 are filled.

The descriptions on the LookML fields are the “why” behind the input. Placed-in-service says it is the day depreciation starts, not the ship date and not the unpack date. A field with a blank description is how two people capitalize on two different days and both think they followed the rule.

EXPECTED. You can point at `reconciliation_rate` in `fact_po_line.view.lkml` and say the other three rates are hidden measures, present so you can show them and then not ship them.

> **WHAT YOU SAY**
> Count distinct on the PO line is the fix for the fan-out. A plain count on this join says 41 lines. There are 40.

5. Build the explore and run reconciliation

WHAT. In Looker, open explore `po_lines`. The base is the PO line. Assets join `left_outer`, relationship `one_to_many`. Program, site, and supplier join `many_to_one`. There is no default filter on received date. Pick program, the reconciliation rate, and filter program to `TS-LINE-RAMP`. On Path A, the same query is `metric_reconciliation_by_program` and `metric_scorecard`.

WHY IT MATTERS. The explore is the only surface a user queries. A Look someone saved in a personal folder is not the definition. The definition is the merged view.

EXPECTED. One row. 40 lines, 38 with an asset, rate 0.95. By program, the other two are 32/32 and 28/28, each 1.0. All programs together, which is what Priya sees with the filter off: 98 / 100 = 98.0%.

> **WHAT YOU SAY**
> I start the explore at the PO line so a line with no asset is still in the denominator. If I start at the asset, the two in-transit lines disappear.

6. Build the five tiles

WHAT. The dashboard file is `lookml/capex_integrity_scorecard.dashboard.lookml`. On Path A, open `dashboard_mock.html`. Five tiles, filters for program and site, drill to the exception list. Default program is `TS-LINE-RAMP`, and that default is visible. It does not hide the in-transit lines, because those lines are inside the program. The site filter is the PO receiving site on the reconciliation tile and the exception tile. On the asset tile and the completeness tile it is the asset site. Say that before you turn it. Person-dependent does not listen to site: Wei alone writes site A, and splitting by site would call the field person-dependent when the program has two editors.

WHY IT MATTERS. Each tile is a query. The owner on the completeness tile is who you ask, which is the change-management part of the job. The same habit, at a larger scale, was 44 requirement rows across 23 systems, each

row pre-wired to an owner before the room saw it. The queue uses the same three classes as that issues log — 331 go-live, 77 fast-follow, 36 enhancement. Those are category counts, not a split of some other total. Every one of these 21 rows is go-live, because the book is wrong without the fix. A nicer chart would be an enhancement, and this queue has none.

TILE	NUMBER
Reconciliation	95.0% (38/40). Alert fires, because 95% is under the proposed 98%.
Exception aging	7, 7, and 7. Age 7 is still 0–7. Age 8 is 8–30. EX-PIS-00017 is age 4.
Completeness	Finance, placed-in-service, 28 filled, 12 missing, 70%. GSM cost center 40/40. TPM program allocation 40/40. AME reuse 12/12. Wei 22/22 sites. Jordan 18/18 sites. Eleven of the twelve blank dates were submitted from site A. The demo row is the one on site B.
First-time-right	52.5% (21/40). The 19 misses are the 12 blank dates, 2 duplicate serials, 3 site mismatches, 1 date-order failure, and 1 orphan.
Person-dependent	3 fields, one editor each, window 2025-12-16 through 2026-03-16.

Click the aging number and you land on `exception_open.csv`: exception id, PO line, asset, field, owner, age, and the sentence. That is the drill. A tile that does not drill is a poster.

EXPECTED. The five numbers above, and you can name the owner of the blank date without opening the tracker. It is Morgan Ellis, Finance, even though Wei or Jordan keyed the asset.

> **WHAT YOU SAY**
> Complete is not valid. Site is 100 percent filled, and three of those codes disagree with the PO. Those three are on the queue, not in the null count.

7. Row-level security, tested as the CM user

WHAT. The model sets `access_filter` on `dim_program.program_id` using user attribute `program_ids`. An admin loads the attribute from `user_program_map.csv`. The user cannot clear it. On Path A, `metric_rls_by_user` is that filter written as SQL.

WHY IT MATTERS. A contract manufacturer does not get the other programs by clearing a dashboard filter. Content folders are not a substitute. The filter has to be in the SQL.

USER	PROGRAMS	LINES VISIBLE	LINES WITH AN ASSET	RATE
Jordan Hale, CM site B	1	40	38	95.0%
Morgan Ellis, Finance	2 (ramp and pack, not the prior program)	72	70	97.22%
Priya Raman, GSM	3	100	98	98.0%

Jordan asking for `PACK-CELL-EXP` returns **0** lines. That is `rls_jordan_pack.csv`. His security is on program, not on site, so he can still see site A's rows of this program. Say that. Site-level restriction is a follow-on, and it is not in this model.

EXPECTED. Jordan sees 95%, not 98%. He cannot produce Priya's blend, and he should not be able to.

> **WHAT YOU SAY**
> I test the access filter as the CM user, and I try to open a program he does not have. Zero rows is the pass. A dashboard filter he can clear is not security.

8. Schedule and alert

WHAT. Two different mechanisms. A schedule sends the dashboard Monday at 07:00 to the owners on the completeness tile — Morgan for the blank dates, not a general list. An alert fires when the certified reconciliation

rate is strictly below 0.98. That 0.98 is rule R4, proposed for this lab and dated before the tile. It is not a number I am attributing to this team. Replace it with theirs when they have one.

WHY IT MATTERS. A schedule does not notice that the rate crossed a line. An alert does not explain the rows. You want both, and you want the datagroup: the cache rebuilds when `max(created_at)` on the asset table moves, and it is never older than 24 hours. A Friday cache read on Monday is how a weekend receipt goes missing. Looker keeps the result cache. The rows stay in the warehouse.

EXPECTED. On `TS-LINE-RAMP`, 0.95 is below 0.98, so `alert_fires` is true. On Priya's unfiltered 0.98, strictly-below is false, so the blend stays quiet. That is the bug you are preventing with the program filter. Morgan's unfiltered book is 0.9722 and would still fire, because she cannot see the prior program that makes Priya's blend land on the line.

> **WHAT YOU SAY**
>
> The Monday email goes to the person who can fill the null. The alert is a separate object, and I do not let an unfiltered blend sit on the line and stay quiet.

9. Fix one record

WHAT. In the system of record, not in the tracker and not on the tile, set the placed-in-service date on the asset for PO line `00017` (`AST-TS-0017`) to `2026-02-17`. That is five days after received `2026-02-12`, so you do not create a new date-order defect. Set `closed_date` on `EX-PIS-00017` to `2026-03-16`. Insert one field edit by `usr_morgan_fin`. Do not update `passed_validation_on_first_entry`. The SQL is `sql/04_rls_and_fix.sql`. Path A already ran it into `expected_outputs/after_fix/`. The CSVs in `data/` are still the open queue. Re-run `00` and `01` if you applied the update and want the open queue back.

WHY IT MATTERS. The write path leaves Looker. The owner fixes the system of record. The next refresh closes the row. If Jordan had typed the date, first-time-right would still not recover, and placed-in-service would pick up a second editor and stop being person-dependent. The fix path is the field owner.

	BEFORE	AFTER
Missing placed-in-service	12	11
Completeness of that field	28/40 = 70%	29/40 = 72.5%
Aging, open	7 / 7 / 7	6 / 7 / 7
Reconciliation	95.0%	95.0%
First-time-right	21/40 = 52.5%	21/40 = 52.5%. The flag on AST-TS-0017 is still 0.
Person-dependent fields	3	3. Morgan is still the only editor of the date.
Alert	fires	fires
EX-PIS-00017	open, age 4	closed 2026-03-16, gone from the open bucket

`after_fix/tracker_vs_fact.csv` shows one linked-asset mismatch. The tracker still has a blank, and the system of record does not. You do not reload the tracker over the correction.

EXPECTED. The table above, from the two scorecard CSVs, not from memory.

> **WHAT YOU SAY**
>
> Closing the exception is the refresh. First-time-right does not move, and I do not want a measure that lets it move.

10. The playbook rule that prevents it next time

WHAT. Write the rule, then compare it to `playbook/RULE-pis-required-at-tagging.md`. A tracker row with a received date and a blank placed-in-service date is quarantined. It does not load. The date cannot be earlier than receipt. Finance owns the definition. The CM coordinator sees the rule in the instructions before the file is due, not in a note after the load.

WHY IT MATTERS. The scorecard found a miss the gate was not asked to catch. Austria was the same shape of work at a different company: data quality about 55% to about 99% within a quarter, by making the rule explicit and staying with it. This lab's first-time-right is 52.5%. I am not saying this file is that program. I am saying the rate moves when the rule is in the gate, and it does not move when someone backfills a cell. On the next tracker, those twelve blanks would be part of the quarantine count. If that pushed the rate over 5%, the decision would be no-go, and that would be the gate working.

EXPECTED. One short rule with an owner, a reject condition, and a sentence that a late fix does not restore first-time-right.

> **WHAT YOU SAY**
>
> The playbook entry is the reject, written down before the next file. The scorecard is how I see it if the rule is not in the gate yet.

Part 3 — Explaining it to the hiring manager

About three minutes. You are walking a model of the work, and you say so. You have not shipped this LookML.

> **THE NARRATION**
>
> A program is still in two trackers. My job is the accuracy of the inputs when it moves onto the system of record, and the change with the GSM, the TPM, the AME, the two CM coordinators, and Finance.
>
> I publish the tolerance before the file shows up. Missing lines against the equipment list: zero. Duplicate line ids: zero. Quarantine under 5 percent. This file is 2 quarantined out of 43, and 40 lines loaded. That is a go. The morning after, I still recount the business rules. Two serials are duplicated, one asset has no PO line, three sites disagree with the PO, one depreciation date is before receipt, and twelve assets have no placed-in-service date. The file gate was not asked to catch those. They are the queue, and each one has an owner.
>
> The model starts at the PO line. Assets join one-to-many, because one line can carry two units. I count distinct line ids. A plain count on this data says 41. There are 40. The certified rate is lines with an asset over lines, 38 over 40, 95 percent. Assets over lines prints 97.5, and a filter that keeps only received lines prints 100. I do not certify either. The definition lives in the view, in Git. A personal report does not get its own formula.
>
> The scorecard is five queries. Reconciliation 95 percent, so the alert fires under the line I proposed at 98, which I would replace with the team's number. Aging is 7, 7, and 7. Completeness by owner: Finance is at 70 percent on the date that starts depreciation, and site is 100 percent filled with three codes that are still wrong. First-time-right is 21 of 40. Three fields were edited by exactly one person in the last 90 days.
>
> I fix one record in the system of record, PO line 00017, the blank date. The exception closes. Missing dates go from 12 to 11. Reconciliation stays 95. First-time-right stays 52.5, because the flag is about the first submit. Then I write the rule that quarantines a blank date on the next file, so the scorecard is not where we notice it the second time.
>
> Row-level security is a user attribute the CM cannot clear. Jordan sees this program only. He sees 95 percent, and a program he does not own returns no rows.

Five follow-ups

Why is the PO line the grain? Because the failure you are managing is a buy with no asset, and an asset-grain table cannot show a row that was never tagged. Two assets on one line are still one line. The orphan is a different check, on the asset, and it does not get to fill the reconciliation hole.

Why Looker rather than Excel for this? Excel holds the rows. It does not hold a definition. The next copy of the file is a second meaning of reconciled, and it will turn `00017` into a number if someone saves it. Looker, if that is the tool they have, keeps the measure in Git and queries the warehouse. Looker Studio can draw the tiles and will not give you that Git history. I would not pretend those are the same product. I have not shipped either model. I have frozen definitions and a parity gate, with SQL Runner named as the surface.

How do you stop two definitions of reconciled? One measure in the view, count distinct on both sides, reviewed like code. The asset version and the received-only version stay in the repo as hidden measures so someone can see the trap, and they are not on the tile. The Monday schedule and the alert both point at the certified measure. A side file that divides assets by lines ordered is a different metric, and it does not replace the tile without a change to the view.

What would you not automate yet? I would not let a model fill placed-in-service, the reuse source, or the program allocation. Those are judgments, and this program's first-time-right is 52.5 percent. I would not train anything on a queue that still has duplicate serials and an orphan. I would automate the gate that is already a rule: trim, parse the date, map the site, reject a blank key, reject an unknown site, reject a line that is not on the equipment list. Capitalization and reuse stay with Finance and the AME until the scorecard says the input is trustworthy.

How does this map to onboarding playbooks? The pilot is this program. The playbook entry is the reject we added after the scorecard found the blank date. The next program inherits the rule, the owner map, the tolerance,

and the same five tiles. What stays local is the equipment list and the CM's site codes, which still have to be mapped. I would ask, on the pilot that became their playbook, which lesson teams actually followed and which one they had to keep enforcing.

Part 4 — Glossary and the checklist

TERM	IN THIS LAB
Dimension	Program, site, owner, a date. Group and filter. Do not sum.
Measure	count_distinct, or a ratio of two. The reconciliation rate is a measure.
Primary key	po_line_id on the line, asset_id on the asset, exception_id on the defect.
Fan-out	Line 00018, one line, two assets. Plain count becomes 2. Count distinct stays 1.
Symmetric aggregate	count_distinct of the primary key. Looker will not let the one-to-many join multiply it. A hand-written ratio is not protected.
Explore	po_lines for reconciliation. exceptions for the queue. assets for first-time-right, so the orphan stays in. completeness and field_edits because those grains must not join into the line.
LookML	The model in lookml/. Production would be a merge. This lab is the files, not a merge you have done.
PDT	exception_aging and person_dependent_fields. Derived tables Looker builds in the warehouse, refreshed by the datagroup. Not stored inside Looker.
Access filter	User attribute program_ids on dim_program.program_id. Jordan cannot take it off.
Datagroup	capex_daily. Trigger on the newest asset timestamp, and never older than 24 hours.
First-time-right	Passed validation on the first submit. A later correction does not flip it.
Person-dependent	A field written by exactly one user in the 90 days ending 2026-03-16, inside the program.
Looker Studio	A different product. Free, no LookML, no Git-backed measure. Fine for the five tiles. Not this model.

Files and the numbers to expect after `python3 run_local.py`. Before the fix unless the path says `after_fix/`.

FILE	EXPECT
data/expected_outputs/go_nogo.csv	43 submitted, 2 quarantined, 40 lines, rate about 0.0465, decision GO
quarantine_rows.csv	00901 unknown site; blank PO line and bad date
tracker_vs_fact.csv	0, 0, and 1 orphan not in the tracker. After the fix: 1 linked-asset mismatch
duplicate_serials.csv	2 serials
orphans.csv	AST-TS-0040
site_mismatches.csv	3 rows
date_logic.csv	1 row, line 00019
referential.csv	orphan 1, everything else 0
reconciliation_by_program.csv	ramp 0.95, pack 1.0, fixture 1.0
reconciliation_traps.csv	0.95 certified; 0.975; 1.0; 1.0
fanout_program.csv	41 joined rows, 40 distinct lines, 39 assets
fanout_line_00018.csv	qty 2, plain count 2, distinct lines 1
missing_pis.csv	ramp 12 of 40. After the fix, 11
completeness_by_field_owner.csv	placed-in-service 28/40. After the fix, 29/40
reuse_denominator_trap.csv	certified 1.0 on 12; trap 0.3 on 40
exception_aging.csv	7, 7, 7. After the fix, 6, 7, 7
first_time_right.csv	21/40 = 0.525, unchanged after the fix

FILE	EXPECT
first_time_right_trap.csv	21/39 = 0.5385
first_time_right_all.csv	81/100 = 0.81 with the filter off. Not the same 81 as any other sketch
person_dependent_fields.csv	3 true. Without the window, program allocation is false (2 editors)
rls_by_user.csv	Jordan 0.95, Morgan 0.9722, Priya 0.98
rls_jordan_pack.csv	0
scorecard.csv	0.95, missing 12, 7/7/7, 0.525, 3 person-dependent, alert true
after_fix/scorecard.csv	0.95, missing 11, 6/7/7, 0.525, 3, alert true
fix_proof_00017.csv	Before: blank date, flag 0, closed blank. After: 2026-02-17, flag 0, closed 2026-03-16
dashboard_mock.html	The five tiles, from the open queue, plus the one-line after
lookml/	The model you would commit. Not a model you have run in Looker unless you did Path B

Regenerate with `python3 generate_data.py && python3 run_local.py` from this directory. If a number in this guide and a CSV disagree, believe the CSV. The SQL is the definition.